The Shell Chronicles

Today's Agenda

01

Introduction to the Bash shell

02

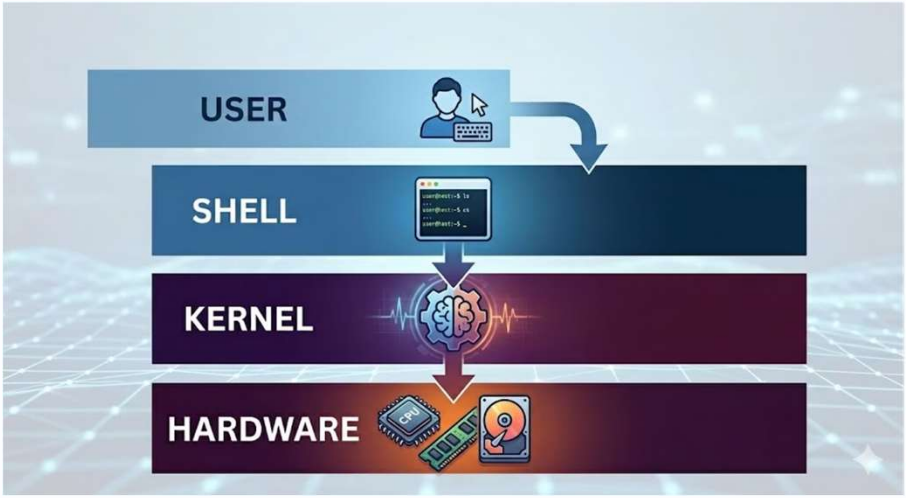
Introduction to restricted shells (and breakouts)

03

Hands-on challenges for restricted shell breakouts

What is a shell?

- **A way to control your computer by typing commands** — like texting instructions instead of pointing and clicking
- **Acts as a translator between you and the computer**, letting you automate tasks and do things a mouse can't



What is Linux?

- **A free operating system** — like Windows or macOS, but open-source and community-built
- **It powers most of the internet**: web servers, cloud platforms, and even Android phones
- **In cybersecurity, most systems we test run Linux** — so we need to speak its language

Meeting the Bash Shell

- Bash is the most common Linux shell
- Commands follow a simple structure:

Syntax:

`command` `argument`

= the action to perform

= what to act on

Example:

```
$ cat  readme.txt
```

Like a sentence: **verb** + **noun** → `cat` + `readme.txt`

Think of it like a sentence: **"Do this action"** + **"to this target"**

Some commands need no target — like asking `pwd` "Where am I?" or `ls` "What's around me?"

Essential Bash Commands

Command	Description
`ls`	List files and directories
`cd`	Change directory — `cd /home`
`cat`	Print file contents — `cat notes.txt`
`pwd`	Print working directory (where you are now)
`sudo`	Run as admin — `sudo -l` lists what you can run

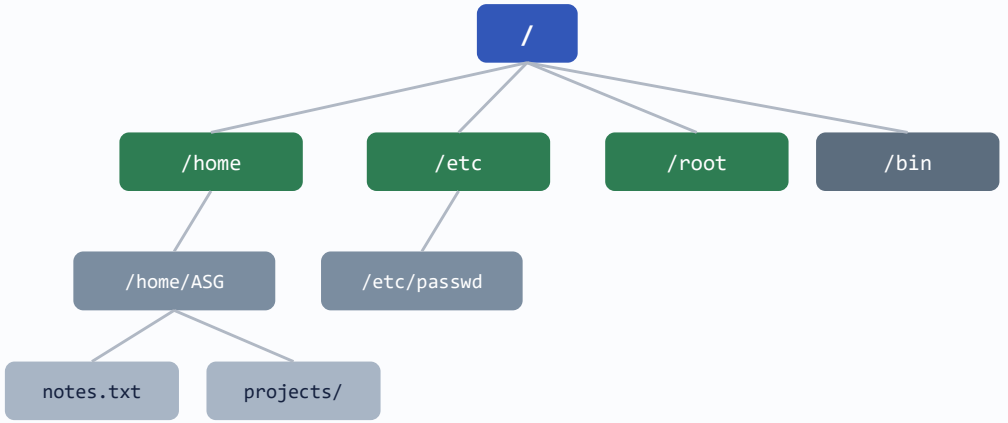
Special Operators

Combine and chain commands together to do more powerful things

Operator	What It Does
`|`	Pipe — send one command's output into another — `cat log.txt | grep error`
`&&`	AND — run next only if first succeeds — `mkdir test && cd test`
`||`	OR — run next only if first fails — `ping host || echo 'down'`
`;`	Semicolon — run commands sequentially — `echo hi ; echo done`
`>`	Redirect — send output to a file — `echo hello > file.txt`

How Files are Organised in Linux

- Linux organises files in a **hierarchical tree** starting from the root
- **Key directories:**
  - `/` — The root (top of the tree)
  - `/home` — Users' personal files
  - `/root` — Admin's private directory
  - `/etc` — System config files



Before We Dive In...

Remember: a **shell** is your command-line interface to talk to the computer. But what if the computer only lets you say certain things?

Think of it like a library computer

- You can browse the catalogue and read books
- But you **can't** install software, change settings, or access the admin panel
- A **restricted shell** works the same way — it's a command line that only allows specific, pre-approved commands

So what happens when someone tries to break out of that cage?

What is a Restricted Shell?

- A shell that limits which commands a user can execute.
- Acts like a digital cage — only **limited** commands are permitted.
- Servers use restricted shells to prevent users from accessing parts of the system they shouldn't see.
- But how robust is the cage? If the shell only checks **part** of what you type, can you sneak something past it?

What Getting Blocked Looks Like

Here's what you'd actually see in a restricted shell:

Allowed Commands

```
$ ls
documents  photos  notes.txt

$ ping google.com
PING google.com: 64 bytes ...
```

✅ These commands work normally

Blocked Commands

```
$ rm notes.txt
restricted: command not allowed

$ ssh admin@server
restricted: command not allowed
```

❌ These commands are blocked

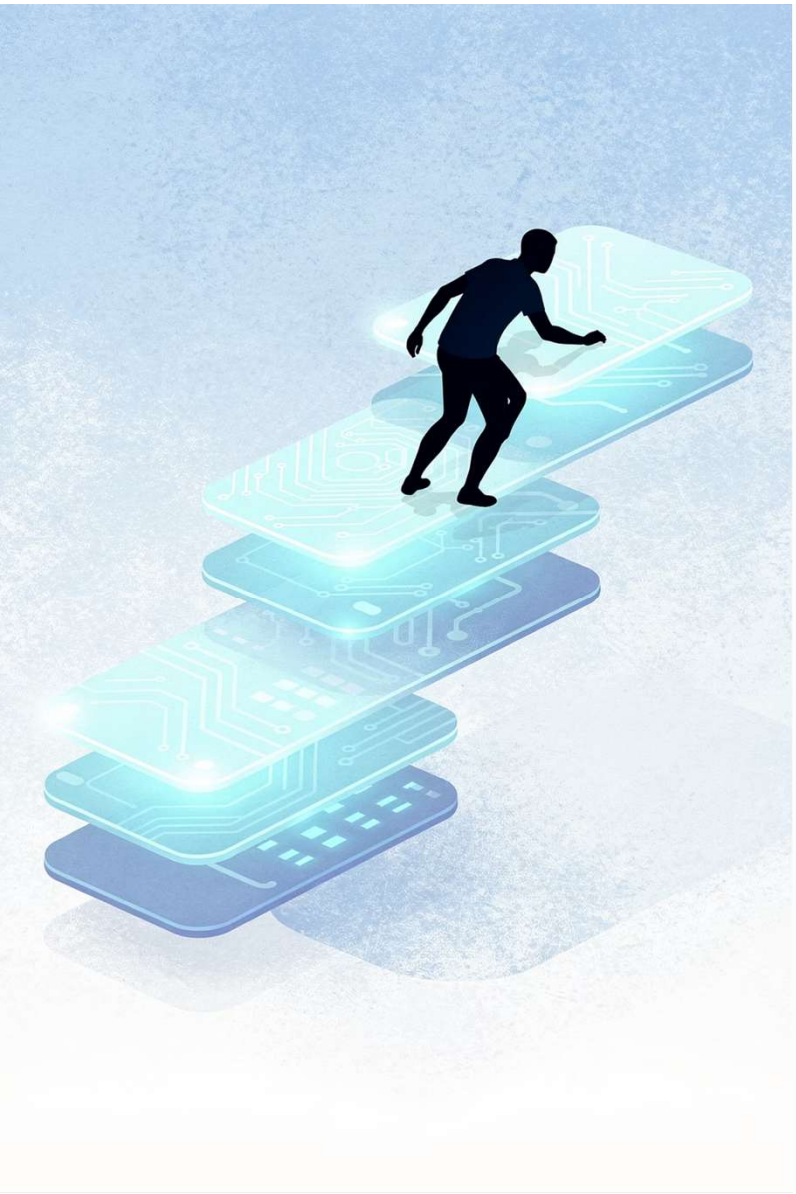
The restricted shell only allows a small set of safe commands. Anything dangerous gets rejected. **Your challenge: find a way around it.**

Why Restricted Shells Matter

- **Principle of Least Privilege:** A security standard where users are given only the minimum level of access needed to perform their job.
- If these restrictions are poorly configured or implemented, they can be bypassed.
- Finding "breakout" techniques to escape the restricted environment and gain full shell access.

Getting Started: Stage 0

- Before starting on the main challenge, go through stage 0 to get familiar with the Bash shell
- Practice the essential commands (ls, cd, cat, whoami) in an emulated terminal.
- Open your browser and navigate to http://localhost to get started
- Follow the on-screen instructions to progress through stage 0
- **For participants with experience with Bash, feel free to skip Stage 0 and begin Stage 1**



The Main Challenge

The Goal

Break through a series of misconfigured security layers.

Flags

Each successful exploitation will print a **Flag** (e.g., ASG{...}) and unlocks the next phase.
No submission of flags needed. Progress gets updated when the flag is printed to the terminal.

Challenge Stages

1

Stage 1 – Path Traversal

The shell is "restricted" to a specific folder. Try breaking out and retrieve the flag from /data/secrets/credentials.txt

2

Stage 2 – Command Injection

The restricted shell limits which commands are available. Break out of the restricted shell and print flag2.txt in the current folder.

3

Stage 3 – Privilege Escalation

Leverage previous command injection to escalate privileges and read the flag at /root/root_flag.txt

Hint: https://gtfobins.org/

4

Stage ? – ???

Look for the **sign** 👀